

BIWEEKLY INTERN PROGRESS REPORT

Document Intelligence & Retrieval Pipeline

Embedding model benchmarks → hybrid retrieval leaderboard → parsing re-evaluation with GLM-OCR

What this report covers

01

Week 5 Recap: Embedding Models

9 dense, sparse, multi-vector, and vision-language embedding models registered and benchmarked

02

Week 5: Hybrid Evaluation Harness

Built a 4-stage retrieval pipeline (BM25 + dense + SPLADE → RRF fusion → LLM judge) and ran it on a 54-pair QA bank

03

Week 6: Revisiting Parsing with GLM-OCR

Benchmarked a new single-model 0.9B VLM against our locked-in multi-stage parsing pipeline

04

Week 7 Plan

Formal CER/TEDS for GLM-OCR, hallucination mitigation, and the remaining embedding-model candidates

Nine models, four representation types

All backends pinned to commit SHAs in models.yaml and benchmarked on the same 54-pair ground-truth QA bank.

✓ Registry locked — 9 models spanning dense, sparse, multi-vector, and vision-language embeddings, evaluated under one harness

DENSE

qwen3-embedding-8b (4-bit)

GO-TO BACKEND

Best overall — top Spec Hit Rate (1.00) and top Faithfulness (0.454) when hybridized with BM25.

SPARSE / LEXICAL

BM25 (custom tokenizer)

GO-TO BACKEND

Custom regex tokenizer preserves full alphanumeric part numbers (e.g. 352952) — the backbone of every hybrid win.

MULTI-VECTOR

bge-m3

GO-TO BACKEND

Only model producing dense + sparse + ColBERT vectors simultaneously — broadest single-model coverage.

From documents to a leaderboard

run_leaderboard.py — four stages, fused with Reciprocal Rank Fusion (k=60)



Embedding cache: 7 min → 20 ms on repeat lookups

tiktoken truncation capped at 4,800 tokens/context

Completion DB avoids re-running Groq generations

HOW EACH METHOD READS THE QUERY — STAGE H, UNPACKED

BM25

Tokenizes the query into terms, scores every chunk by term-overlap statistics — no vectors, just counting.

Dense

Runs the query through the same embedding model used on chunks, then ranks by cosine similarity.

SPLADE

Runs the query through its own network into a sparse query vector, scored via dot product.

Top model / strategy combinations

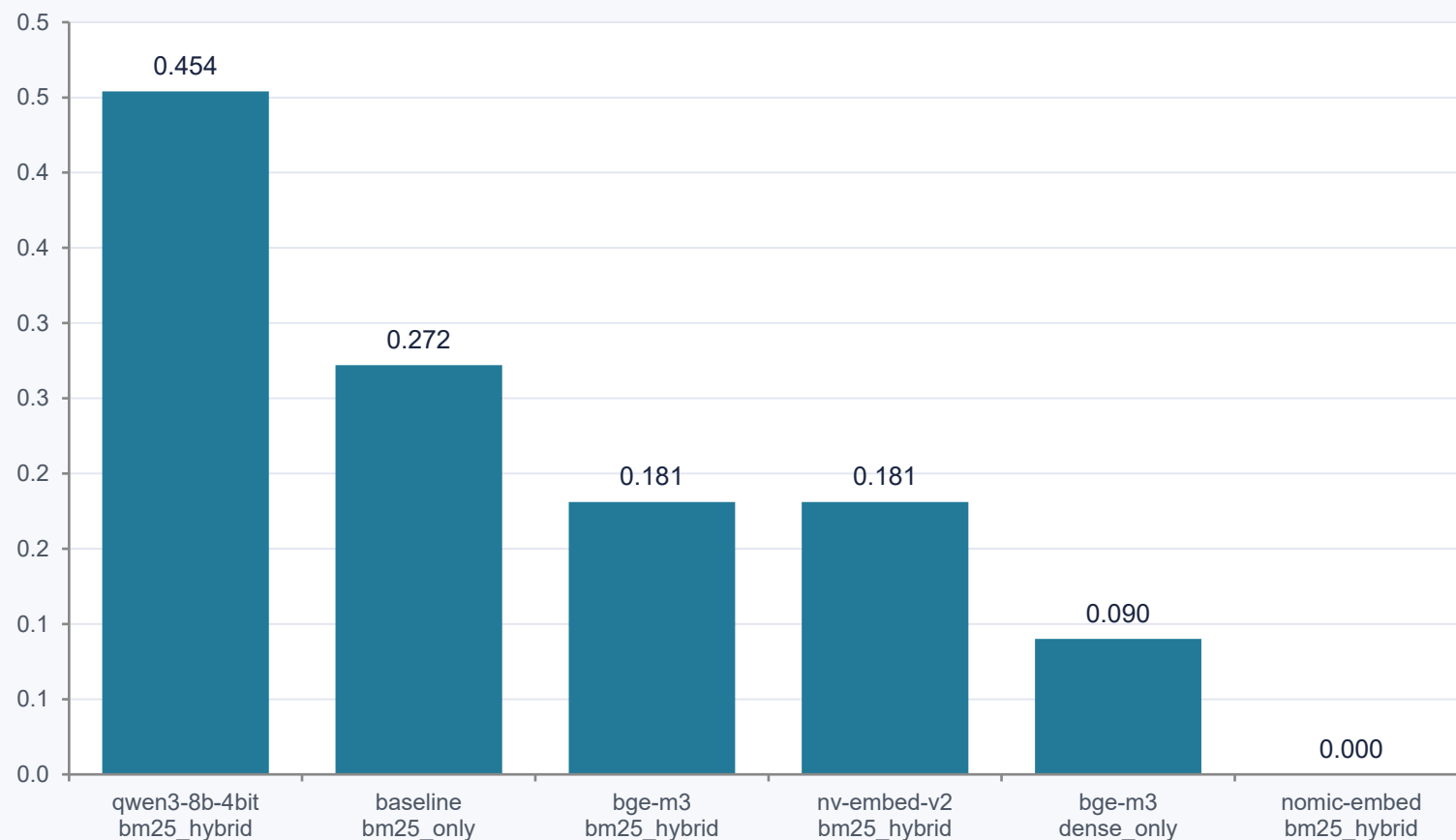
Scored across all 54 Q&A pairs — Spec Hit Rate, Overall Hit Rate, and downstream generator Faithfulness

Rank	Model	Strategy	Spec Hit@5	Overall Hit@5	Faithfulness
1	qwen3-embedding-8b-4bit	bm25_hybrid	1.00	0.518	0.454
2	baseline	bm25_only	1.00	0.722	0.272
3	bge-m3	bm25_hybrid	1.00	0.444	0.181
4	nv-embed-v2-fp16	bm25_hybrid	1.00	0.500	0.181
5	bge-m3	dense_only	1.00	0.240	0.090
6	nomic-embed-text	bm25_hybrid	1.00	0.462	0.000
7	bge-m3	dense_sparse_colbert_hybrid	0.50	0.166	0.090
8	granite-vision-embedding	text_only	0.50	0.363	0.000

qwen3-embedding-8b-4bit + bm25_hybrid ranks #1 on Faithfulness (0.454) — exact-match lexical context prevents the generator from hallucinating spec values.

Faithfulness by model / strategy

Downstream RAG-generator faithfulness (Ragas), for the six models with a perfect Spec Hit Rate@5



READING THE CHART

- All six arms hit a perfect 1.00 Spec Hit Rate@5
- Faithfulness is where they separate — hybrid BM25 arms dominate
- nomic-embed-text scores 0.000 despite perfect retrieval — a generation-side gap, not a retrieval one
- Exact-match lexical context is the strongest predictor of faithful answers

What the benchmark tells us

1.00 hit rate

Lexical BM25 wins on spec codes

Subword tokenizers fragment part/model numbers (e.g. 352952). A regex tokenizer preserving full alphanumeric strings hits 100% Hit Rate@5 on spec lookups.

RRF k=60

Hybrid fusion is essential

Neither pure dense nor pure sparse retrieval covers every query type. RRF-fused hybrid retrieval gives the best accuracy and resilience across mixed workloads.

0.454 best

Exact context → higher faithfulness

Downstream LLM faithfulness rises when exact-match lexical snippets sit in the context window, preventing the generator from hallucinating spec values.

3-way coverage

Dense, sparse & ColBERT are complementary

Dense models excel at abstract synthesis; sparse/multi-vector excel at token-level precision. bge-m3 combines all three for full coverage.

GLM-OCR: a single-model contender

zai-org/GLM-OCR (Zhipu AI) — a compact vision-language model that reads pixels and writes structured Markdown directly, no separate stages.

0.9B

PARAMETERS

vs. an 885M-param layout model alone in the old pipeline

94.62

OMNIDOCBENCH V1.5

#1 on the public leaderboard at release

1 model

REPLACES 4–5

layout + OCR + table + caption + cleanup stages, unified

INITIAL PIPELINE — 4–5 CHAINED MODELS

DocLayoutYOLO → EasyOCR / Tesseract → Docling + TableFormer/TATR → Qwen2.5VL-3B captioning → LLM cleanup pass

GLM-OCR — ONE UNIFIED VLM

Vision encoder + causal LM with Multi-Token Prediction — outputs structured Markdown (tables, headers, layout) directly from page pixels.

Text OCR: page-level results

Real GPU inference, cuda:0, GTX 1650, 4-bit quantized — scored end-to-end on full, un-cropped 300 DPI pages

Page	GT Chars	Pred Chars	CER	WER	Latency
1 (Cover)	416	478	0.8341	0.8667	105.6s
2 (Diagnostics Catalog)	1,734	1,541	0.4683	0.5882	148.9s
4 (Ordering & Contact)	1,563	1,442	0.4210	0.6468	134.3s
Filtered Average (1, 2, 4)	3,713	3,461	0.4894	0.6441	129.6s / page

Excluded: Page 3 — GLM-OCR entered a repetition loop, generating 3,831 chars of table content vs. 1,076 GT chars ($\approx 3.5\times$ repeat). See Slide 11.

Not apples-to-apples yet: baseline EasyOCR CER (0.1078) was scored on 64 pre-cropped element bounding boxes. GLM-OCR CER (0.4894) is end-to-end on full uncropped pages — it also absorbs line-ordering, header/footer, and Markdown-formatting differences the cropped baseline never sees. My guess is the configuration of GLMOCR and the baseline models are different, as the eval criteria is based on baseline, the CER is coming higher, I will look into it next week.

Table extraction: GLM-OCR takes the lead

Evaluated with GLM-OCR's official “Table Recognition:” task prompt, which activates its structured HTML decoder head

Model	TEDS	TEDS (Structure)	Cell F1
Docling (TableFormer)	0.7295	0.7295	0.9828
TATR (Table Transformer)	0.7444	0.8684	0.4213
GLM-OCR (official prompt)	0.9996	1.0000	1.0000

0.9993

e0043 — Page 3

Patient Monitor Feature Comparison (14×4)

1.0000

e0074 — Page 4

ServicePlus Plans (4×3)

GLM-OCR beats every baseline table specialist — 0.9996 TEDS vs. 0.7444 (TATR), 0.7295 (Docling).

Bring Unlimited OCR into the comparison, then re-run the harness

Same protocol, one more model — then the winning parser feeds directly into the Week 5 retrieval pipeline

1

Benchmark Unlimited OCR

Run the same page-level CER/WER + official-prompt TEDS protocol on Unlimited OCR for a fair three-way comparison

2

Fix the repetition loop

Add generation stop-criteria / max-new-tokens limits to GLM-OCR before it touches dense, table-heavy pages

3

Swap the baseline in the harness

Re-run the Week 5 retrieval pipeline with GLM-OCR and Unlimited OCR replacing DocLayoutYOLO + EasyOCR + Docling

4

Compare end-to-end faithfulness

Measure whether better parsing accuracy actually lifts RAG faithfulness downstream, not just parsing-stage metrics

THANK YOU

Questions & Discussion

Next check-in: Week 7 — Unlimited OCR benchmark, and a full harness re-run with GLM-OCR + Unlimited OCR replacing the baseline parser